

Pythonの開発に ハーネス適用を 検討する



AI駆動開発 Women's Party!
2026/05/24
yuuuka@PyLadies Tokyo

自己紹介@yuuuka

- 文系20卒
- 某外資系企業のエンジニア
- PyLadies Tokyo staff(since 2023/06)
- Like:Python,AWS,猫etc...



初同人誌寄稿

ゆめみより '26 「ゆめみより製作委員会」合同技術書

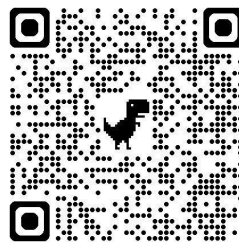


EC2で Pythonのデータ分析ライブラリを比較した話

Kiroの有料サブスクリプションを無料で適用した話

0円

でダウンロードできます



Women's Base 1周年
おめでとうございます 🎉





About PyLadies

女性Pythonistaのための国際的コミュニティ

コミュニティ, イベント,
ソーシャルネットワークを介した交流を通じて
Pythonを学ぶ女性仲間をつなげ,
お互いに助けあうネットワークを作る

<http://www.pyladies.com/about>



PyLadies Tokyo

2014年9月に発足した
日本初のPyLadies支部
月1程度のMeetupを
メインに活動中
(東京を中心に活動)

<https://pyladiestokyo.github.io/>

本日は話すこと

- ①ハーネスエンジニアリングとは？
- ②Python開発環境に対してハーネスとして何ができるか検討する

ハーネスエンジニアリングとは

「ハーネスエンジニアリング」はいつ登場したのか？

**2026年
2月11日**

OpenAI / Ryan Lopopolo

openai.com/index/harness-engineering/

「開発パフォーマンスが大幅に向上した」という紹介のみ。具体的に何をしたかは書かれていなかった。

「初期の進捗は予想より遅れましたが、それは AIに能力がなかったからではなく、環境が十分に定義されていなかったためです」

その後バズワードとして度々聞くことになったが、今も定義がバラバラ

→「ハーネスとは何か」をそれぞれが解釈し、記事・登壇 OSSで独自の定義を発信し続けている状態

「ハーネス」と聞いて最初に浮かんだイメージ

最初のイメージ: 制御する・束縛する・抑える

- mainブランチへの直接プッシュを止める
- rm -rf の実行を防ぐ
- 意図しないファイル削除を防止する

でも実は...

能力を最大限に引き出すための装具



「ハーネス」とははなんなのか？

定義は人によってさまざま — 同じ言葉でも指している範囲が違う

コンテキスト設計の話として語られる

出力検査・ガードレールの話として語られる

ドキュメントやコードの劣化対策として語られる

「なんとなく便利な仕組み」の総称になりがち

なぜバラバラなのか？ → AIを「使う人と作る人、扱うスコープの広さによって、見えているものが違うから。

各提唱者の定義

提唱者	定義・キーワード	立場
① Mitchell Hashimoto	ミスを繰り返さない仕組み フィードバックループ	ユーザー
② Ryan Lopopolo(OpenAI)	環境全体の設計 Humans steer. Agents execute.	ビルダー
③ Prithvi Rajasekaran (Anthropic)	Planner–Generator–Evaluator 長時間タスク設計	ビルダー
④ Vivek Trivedy(LangChain)	Agent = Model + Harness	ビルダー

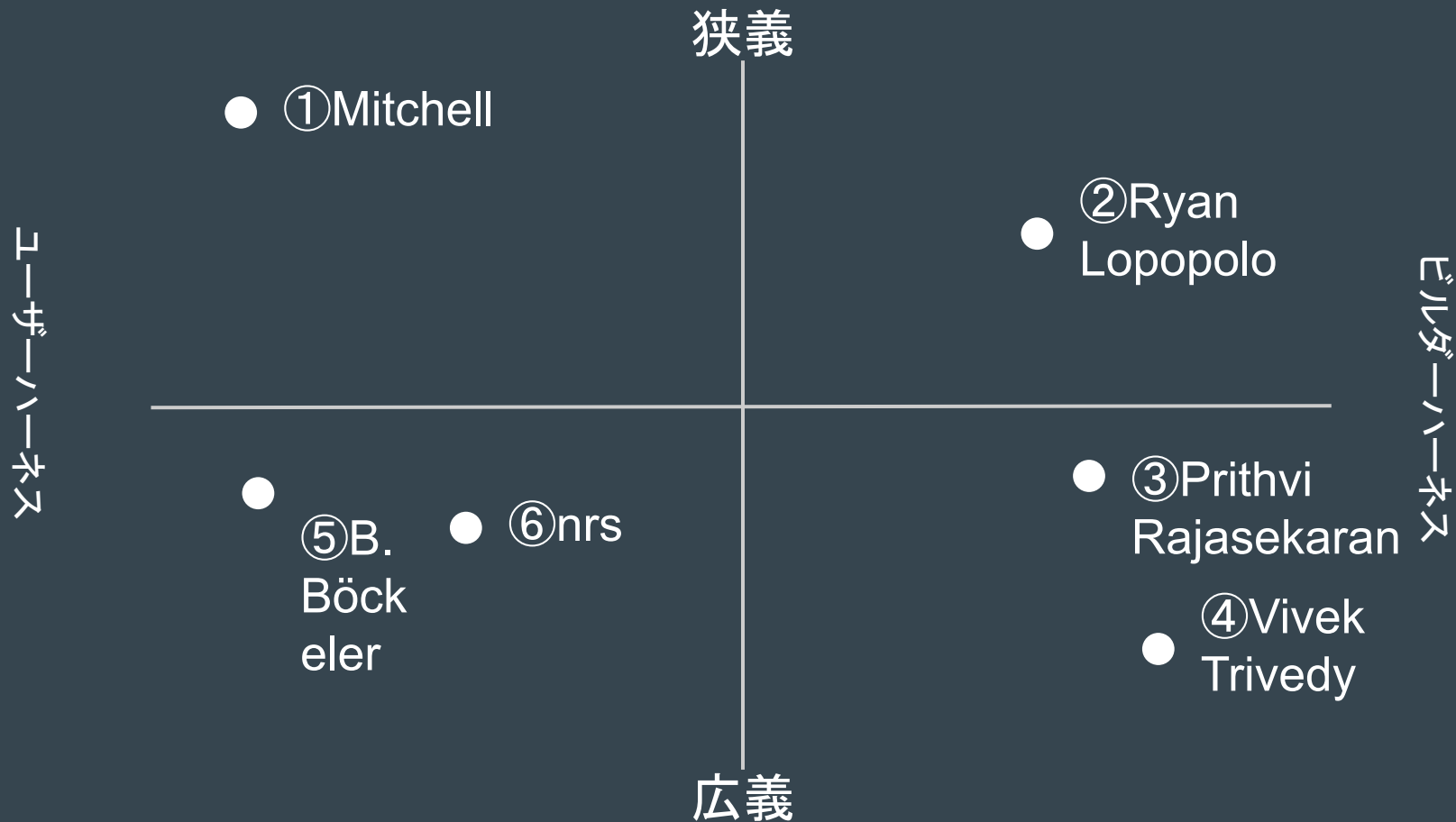
各提唱者の定義

提唱者	定義・キーワード	立場
★ ⑤ Birgitta Böckeler	Guides + Sensors ステアリンググループ / 3カテゴリ	ユーザー
⑥ nrs (成瀬允宣)	モデルの外側で制御・誘導・検証 する仕組みの総称	ユーザー /ビルダー

★ おすすめ:

Guides/Sensorsの枠組みが最も体系的。ユーザー目線で書かれており入門に最適。

定義の整理: 2つの軸



Computational vs Inferential

	Computational(決定論的)	Inferential(確率論的)
実行主体	CPU・高速・信頼性高い	GPU / NPU・低速・非決定論的
特徴	安価に繰り返し実行可能 結果が確実	意味的判断が可能 非決定論的
フィードフォワード例	LSP・型チェッカー スクリプト	AGENTS.md・Skills コーディング規約
フィードバック例	テスト・linter・ruff 構造解析・型チェック	AIコードレビュー LLM as judge

出典: Birgitta Böckeler「Harness engineering for coding agent users」(martinfowler.com)

ハーネスの構造

フィードフォワード

動く前に方向づける

- 役割と目的を明示する
- 背景知識をあらかじめ渡す
- 完了条件を事前に合意する
- 使える手段を必要な範囲に絞る



エージェント
実行

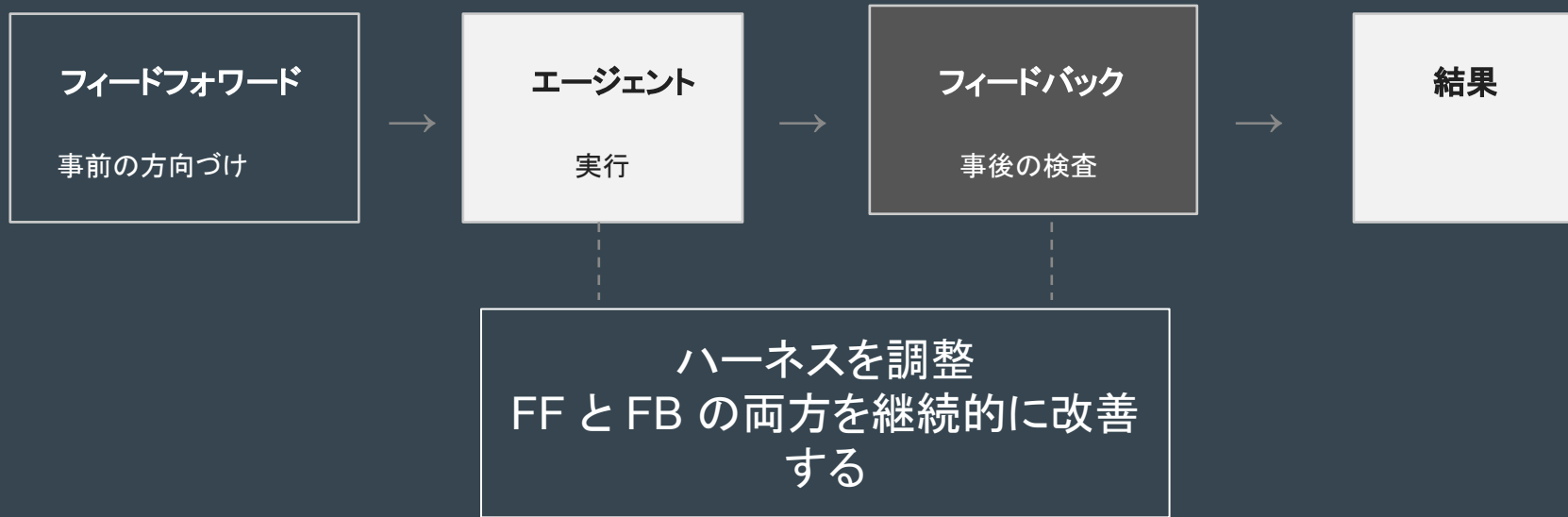
フィードバック

動いた後に自己修正を促す

- 出力を検査しやすい形で受ける
- 異なる観点から複数回チェック
- 不備があれば修正を促す
- 停滞したら別の手立てへ

ステアリンググループ

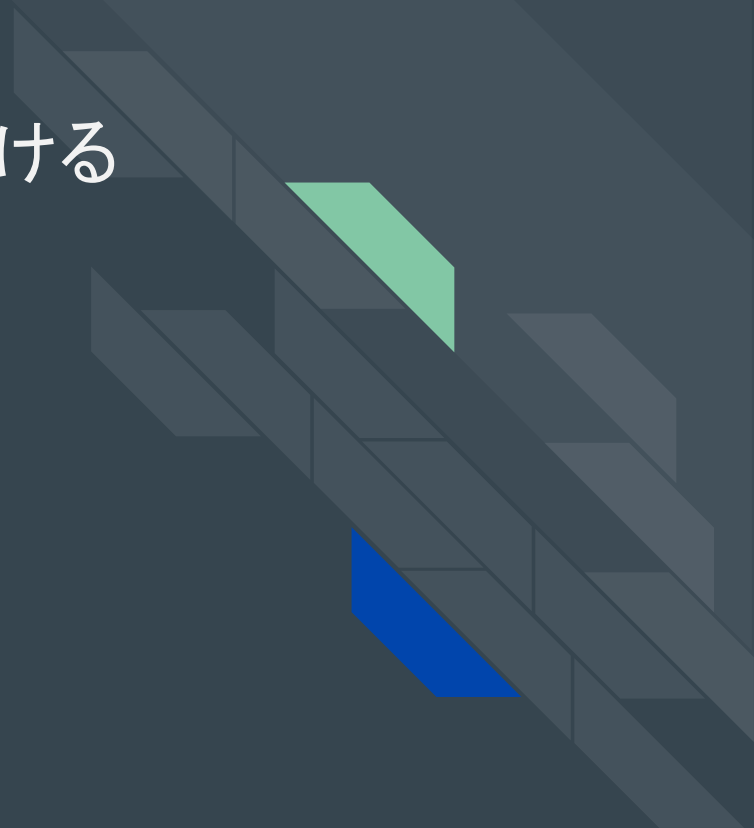
人間の役割:「書く」から「操縦する」へ



Human (Steering)

AIが生成した結果を見て、GuidesとSensorsを継続的に改善する。コードを「書く人」から、エージェントを「操縦する人」へ。ハーネスを育て続けることが新しいエンジニアリング。

私が考えるPythonでの開発における ハーネスの検討





私が考えるハーネス

AIが能力を発揮できる 環境づくり

スコープ: モデル以外の全要素(広義)

立場: AIツールを使う人

本質: 抑制ではなく能力発揮の環境整備

フィードフォワード

AIが迷わず安全に動ける「クリーンな土台」を構築する

1. ガイドファイル

AGENTS.md / CLAUDE.md
Kiro steering files

- Python規約・禁止パターンを明記
- 日本語での出力・コメントを指定
- Permissionsのallow/denyで操作範囲を制限

2. 環境の完全隔離(どこに)

uv venv

- 仮想環境を強制しグローバルと完全に分離
- AIが勝手にグローバルを変更できなくなる
- *uv sync*で環境を瞬時に再現

3. 依存関係の固定(何を)

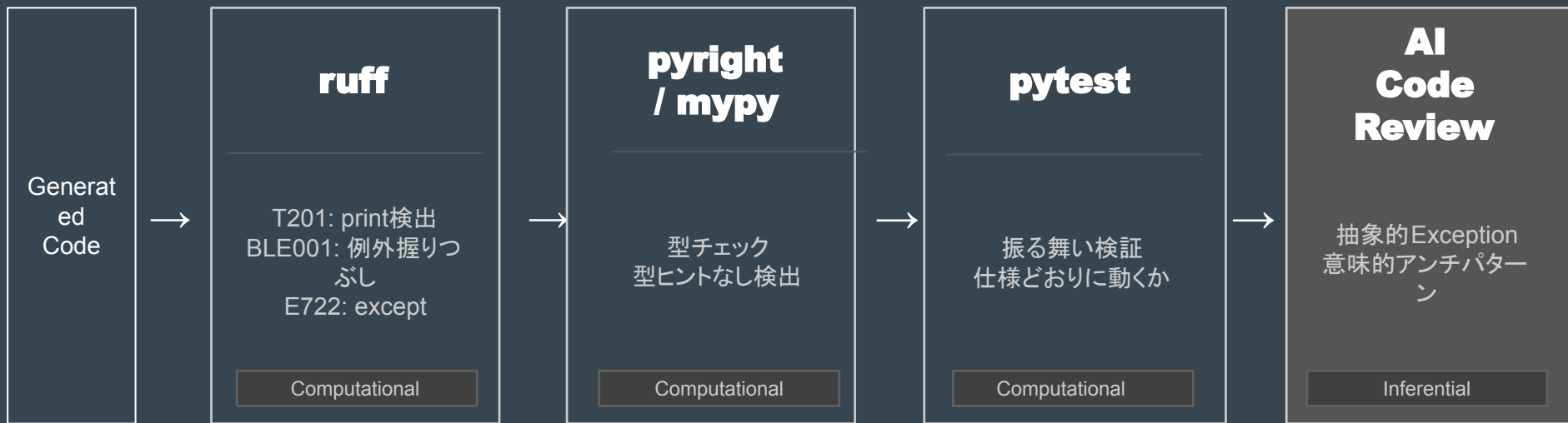
uv.lock

- バージョンを厳格に固定
- AIによる未承認パッケージの追加を防止
- PyPIサプライチェーン攻撃対策

型ヒント・docstring | AIへのData Context層として機能。書くほどエージェントの推論精度が上がる

フィードバック

AIのミス人間が指摘するのではなく、ツールが瞬時に弾き返す



pyproject.toml : 上記すべてのツール設定を一元管理。ハーネスの設定散逸を防ぐ
※デッドコード検出には vulture も併用推奨

自分のプロジェクトに当てはめてみた



できていること

Specs (仕様駆動開発)

Guide / FF

Kiro

Steering files

Guide / FF

Kiro

Agent Hooks

Sensor / FB

Kiro

CLAUDE.md

Guide / FF

Claude Code



足りていないこと

Permissionsのallow/deny

Sensor / FF

● 最優先

Skills / Subagents

Guide / FF

● 高

ruff・pyright・pytest

Sensor / FB

● 高

ガイド側はある程度整備できているが、センサー側がまだ。ハーネスは常に育て続けるもの。

明日からできること

ハーネスは「開発者の経験」の外部化

01

uv で環境を整える

仮想環境の分離から。

グローバル汚染を防ぎAIが安全に動ける土台を作る。

```
uv venv
uv sync
```

02

ruff を **pyproject.toml** に設定する

linter と formatter を一本化。print・例外握りつぶしをComputational に検出する。

```
[tool.ruff.lint]
select = ["T201",
"BLE001", "E722"]
```

03

AGENTS.md にプロジェクトの文脈を書き出す

Python規約・禁止パターン・日本語指定を明記。
エージェントへのフィードフォワードの起点。

```
# AGENTS.md
Zen of Python
禁止パターン
日本語で応答
```

AIと並走しよう！